

סטטוס פיתוח והצגת שאלונים

שיתוף פעולה עם מחלקה לנפגעי ראש
במכון פוירשטיין

פיתוח וניהול הפרויקט: איתמר בבאי ורז אוננו

מרצה מנחה: פרופסור אראל סגל

ניהול הפרויקט מטעם מכון פוירשטיין: מאיה בן שמחון, מרפאה בעיסוק

הדרגתיות וסיגול חזרתיות

עלה קושי בקצב המשחק הראשוני. הפתרון המיושם מתבסס על הדרגתיות "מיקרו-רמתית":

✓ **תוספת אתגר בודד:** כל תת-רמה מוסיפה אלמנט קוגניטיבי אחד בלבד

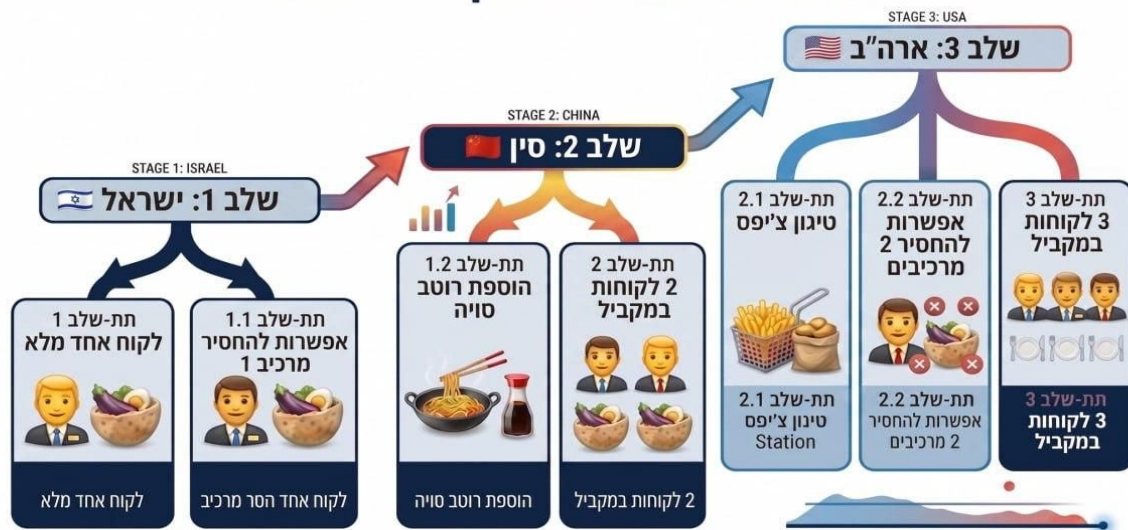
✓ **בנייה נדבכית:** כל רמה מתבססת על הקודמת לחיזוק הביטחון

✓ **הסתגלות סביבתית:** במעברים בין

מדינות (סין, ארה"ב) יוצג סרטון מעבר

ליעד החדש

מפת דרכים שלבי המשחק - עליית מדרגה



התאמת קצב המשחק

בקליניקה יש מנעד רחב של מטופלים ברמת תפקוד שונה ועולה צורך בהתאמת קצב המשחק.

לוח בקרה

זמן לסיום השלב - 30 שניות - 30 שניות

מד סבלנות - 7 sec

סימון פריטים - ☐ הפעלת לכלוך - ☒

מס' לקוחות במקביל - 3 2 1

מס' מרכיבים בשלב - 6/6

שמור איפוס

זיכרון עבודה

- סימון רכיבים שכבר נוספו ב-X בבועה.
- אפשרות לבחירת מספר רכיבים מצומצם לרמה (הסרה 3-4 מהרכיב החדש ביותר)
-

עומס קוגניטיבי

- הסרת הלכלוך מהמסך.
- מספר הלקוחות במקביל 1/2/3 כאשר המקסימלי תואם לדרגת הקושי של הרמה.

ניהול זמן

- הוספת/החסרת 30 שניות.
- קביעת זמן מד הסבלנות של לקוח בין 7-12 שניות.

לכלוך מסך מהרטבים

```
1 reference
private void SetupDirtToggle()
{
    if (showDirtToggle == null)
        return;

    showDirtToggle.isOn = DirtEnabled;
}
```

```
1 reference
private void ApplyDirtSetting()
{
    if (showDirtToggle == null)
    {
        DirtEnabled = true;
        Debug.LogWarning("[ControlPanelUI] Show Dirt Toggle is not assigned. Dirt remains enabled.");
        return;
    }

    DirtEnabled = showDirtToggle.isOn;

    if (!DirtEnabled && DirtStateManager.Instance != null)
        DirtStateManager.Instance.Clean();

    Debug.Log($"[ControlPanelUI] Dirt enabled saved: {DirtEnabled}");
}
```

מד מצב רוח של הלקוחות

```
1 reference
private void SetupAngerTimeSlider()
{
    if (angerTimeSlider == null)
        return;

    angerTimeSlider.minValue = 7;
    angerTimeSlider.maxValue = 12;
    angerTimeSlider.wholeNumbers = true;
    angerTimeSlider.value = CustomerMoodTimer_levels.RuntimeSecondsPerStage;
    angerTimeSlider.onValueChanged.AddListener(OnAngerTimeSliderChanged);
}
```

```
1 reference
private void ApplyAngerTimeSetting()
{
    if (angerTimeSlider == null)
        return;

    int selectedAngerTime = Mathf.RoundToInt(angerTimeSlider.value);
    CustomerMoodTimer_levels.SetRuntimeSecondsPerStage(selectedAngerTime);

    Debug.Log($"[ControlPanelUI] Anger time saved: {selectedAngerTime} seconds per stage.");
}
```

סימון רכיבים בבועה

```
1 reference
private void SetupMarkAddedItemsToggle()
{
    if (markAddedItemsToggle == null)
        return;

    markAddedItemsToggle.isOn = MarkAddedItemsEnabled;
}
```

```
1 reference
private void ApplyMarkAddedItemsSetting()
{
    if (markAddedItemsToggle == null)
    {
        MarkAddedItemsEnabled = false;
        Customer.ClearAllCustomerIngredientMarkers();
        Debug.LogWarning("[ControlPanelUI] Mark Added Items Toggle is not assigned.");
        return;
    }

    MarkAddedItemsEnabled = markAddedItemsToggle.isOn;

    if (!MarkAddedItemsEnabled)
        Customer.ClearAllCustomerIngredientMarkers();

    Debug.Log($"[ControlPanelUI] Mark added items saved: {MarkAddedItemsEnabled}");
}
```

מספר רכיבים

```
1 reference
private void SetupIngredientCountSlider()
{
    if (ingredientCountSlider == null)
        return;

    ingredientCountSlider.Slider.SliderEvent Slider.onValueChanged { get; set; }
    ingredientCountSlider.onValueChanged.AddListener(OnIngredientCountSliderChanged);
}
```

```
1 reference
private void ApplyIngredientCountSetting()
{
    LevelIngredientAvailabilityManager manager = LevelIngredientAvailabilityManager.Instance;

    if (manager == null)
    {
        Debug.LogWarning("[ControlPanelUI] LevelIngredientAvailabilityManager.Instance was not found.");
        return;
    }

    selectedIngredientCount = Mathf.Clamp(
        selectedIngredientCount,
        manager.MinIngredientCount,
        manager.MaxIngredientCount
    );

    manager.ApplyIngredientCount(selectedIngredientCount,
        SyncIngredientCountSliderFromManager());

    Debug.Log($"[ControlPanelUI] Ingredient count saved: {selectedIngredientCount}/{manager.MaxIngredientCount}");
}
```

```
4 references
private void SyncIngredientCountSliderFromManager()
{
    LevelIngredientAvailabilityManager manager = LevelIngredientAvailabilityManager.Instance;

    if (manager == null)
    {
        minIngredientCount = 1;
        maxIngredientCount = 1;
        selectedIngredientCount = 1;

        if (ingredientCountSlider != null)
        {
            ingredientCountSlider.interactable = false;
            ingredientCountSlider.minValue = 1;
            ingredientCountSlider.maxValue = 1;
            ingredientCountSlider.SetValueWithoutNotify(1);
        }

        UpdateIngredientCountText();
        return;
    }

    minIngredientCount = manager.MinIngredientCount;
```

מספר לקוחות בו-זמנית

4 references

```
private void UpdateCustomerLimitButtonsVisualState()
{
    UpdateCustomerLimitButton(customerLimit1Button, 1);
    UpdateCustomerLimitButton(customerLimit2Button, 2);
    UpdateCustomerLimitButton(customerLimit3Button, 3);
}
```

3 references

```
private void UpdateCustomerLimitButton(Button button, int value)
{
    if (button == null)
        return;

    bool supported = value <= supportedCustomerLimit;
    bool selected = value == selectedCustomerLimit;

    button.interactable = supported;

    Color targetColor;

    if (!supported)
```

1 reference

```
private void ApplyCustomerLimitSetting()
{
    CustomerManager manager = CustomerManager.Instance;

    if (manager == null)
    {
        Debug.LogWarning("[ControlPanelUI] CustomerManager.Instance was not found.");
        return;
    }

    int supported = manager.GetMaxSupportedConcurrentCustomers();
    selectedCustomerLimit = Mathf.Clamp(selectedCustomerLimit, 1, supported);

    manager.SetMaxConcurrentCustomers(selectedCustomerLimit);
    SyncCustomerLimitButtonsFromManager();

    Debug.Log($"[ControlPanelUI] Customer limit saved: {selectedCustomerLimit}");
}
```


הוספת / צמצום זמן

2 references

```
private void OnAdd30SecondsClicked()
{
    AddTimeToCurrentLevel(30f);
}
```

2 references

```
private void OnRemove30SecondsClicked()
{
    AddTimeToCurrentLevel(-30f);
}
```

2 references

```
private void AddTimeToCurrentLevel(float seconds)
{
    ResolveLevelTimerIfNeeded();

    if (levelTimerBehaviour == null || addTimeMethod == null)
    {
        Debug.LogWarning("[ControlPanelUI] No level timer with AddTimeSeconds(float) was found or assigned.");
        return;
    }

    addTimeMethod.Invoke(levelTimerBehaviour, new object[] { seconds });
}
```

2 references

כפתורי שמירה ואיפוס

2 references

```
public void ResetSettings()
{
    if (angerTimeSlider != null)
        angerTimeSlider.value = defaultAngerTimeSeconds;

    if (markAddedItemsToggle != null)
        markAddedItemsToggle.isOn = defaultMarkAddedItemsEnabled;

    if (showDirtToggle != null)
        showDirtToggle.isOn = defaultDirtEnabled;

    MarkAddedItemsEnabled = defaultMarkAddedItemsEnabled;
    DirtEnabled = defaultDirtEnabled;

    ResetCustomerLimitToLevelDefault();
    ResetIngredientCountToLevelDefault();

    if (!MarkAddedItemsEnabled)
        Customer.ClearAllCustomerIngredientMarkers();

    if (!DirtEnabled && DirtStateManager.Instance != null)
        DirtStateManager.Instance.Clean();

    UpdateAngerTimeText();
}
```

4 references

```
private void SyncIngredientCountSliderFromManager()
{
    LevelIngredientAvailabilityManager manager = LevelIngredientAvailabilityManager.Instance;

    if (manager == null)
    {
        minIngredientCount = 1;
        maxIngredientCount = 1;
        selectedIngredientCount = 1;

        if (ingredientCountSlider != null)
        {
            ingredientCountSlider.interactable = false;
            ingredientCountSlider.minValue = 1;
            ingredientCountSlider.maxValue = 1;
            ingredientCountSlider.SetValueWithoutNotify(1);
        }

        UpdateIngredientCountText();
        return;
    }

    minIngredientCount = manager.MinIngredientCount;
}
```

Dashboard - מנגנון איסוף והצגת נתונים



3. גיבוי (Cloud Sync)

סנכרון אוטומטי ל-Unity Cloud לצורך גיבוי נתונים מאובטח והבטחת רציפות במשחק למשתמש חוזר, מאותו מקום שעזב את המשחק.



2. שמירה ב - Supabase

שמירה אוטומטית של נתוני המחקר ל-Supabase — מסד נתונים מאובטח בענן המשמש כמקור הנתונים לדשבורד.



1. איסוף נתונים

תיעוד כל רמה (LevelAttempt): הצלחות, שגיאות, אינהיביציה, לחיצות כפולות, זמנים, הגדרות לוח הבקרה וסדר המשחק של השחקן.



4. ניתוח והצגת נתונים

דשבורד web: קובץ HTML יחיד עם CSS ו-JavaScript מוטמעים, נגיש דרך הדפדפן (GitHub Pages). כולל גרפים (Chart.js), סינון לפי מטופל ורמה, השוואה בין סשנים, הגדרות לוח הבקרה לכל רמה והוספת הערות מרפאה.

URL: https://gamedev000.github.io/Sababich/ExternalDashboard/sababich_research_dashboard.html

מנגנון איסוף והצגת הנתונים - Dashboard

דפסה / שמור PDF

ט רען

דשבורד מחקר - סבכיר

מטופל:

דשבורד

סינון

מתאריך:

dd/mm/yyyy

עד תאריך:

dd/mm/yyyy

רמה:

הכל

תוצאה:

הכל

אפס סינון

סשנים3

ממוצע מטבעות331

זמן ממוצע להכנת מנה (שנ')8.9

מנות שהוגשו סה"כ158

שגיאות סה"כ26

היסטוריית סשנים

מיון:

אחרון ראשון

דשבורד

11:12 29.5.2026

עבר

מטבעות: 958

רמה 2,2, רמה 3

דשבורד

10:58 29.5.2026

עבר

מטבעות: 1723

רמה 1, רמה 1,1, רמה 1,2, רמה 2, רמה 2,1

דשבורד

09:55 29.5.2026

עבר

מטבעות: 1957

רמה 1, רמה 1,1, רמה 1,2, רמה 2, רמה 2,1, רמה 3

מנגנון איסוף והצגת הנתונים - Dashboard

מיון: אחרון ראשון

דשבורד11:12 29.5.2026עברמטבעות: 958רמה 2,2, רמה 3

דשבורד10:58 29.5.2026עברמטבעות: 1723רמה 1, רמה 1.1, רמה 1.2, רמה 2, רמה 2.1

דשבורד09:55 29.5.2026עברמטבעות: 1957רמה 1, רמה 1.1, רמה 1.2, רמה 2, רמה 2.1, רמה 2.2, רמה 3

פרטי ששן

תאריך: 11:12 29.5.2026 מטופל: דשבורד

רמה	תוצאה	מטבעות	זמן (שנ")	הוגש סה"כ	מושלם	שגוי	קליקים כפולים	ילד גלוטן הופיע	הוגש גלוטן	לא הוגש גלוטן	זמן הכנה ממוצע	
רמה 2.2	עבר	445	108	14	13	1	0	3	0	3	8.3	
רמה 3	עבר	513	84	17	16	1	1	2	0	2	5.3	

מנות שהוגשו

רמה 2.2	רמה 3
לקוחות שהגיעו	30
לקוחות שלא קיבלו מנה	13
הוגש סה"כ	17
מושלם	16
שגוי	1

מנגנון איסוף והצגת הנתונים - Dashboard

מיון:

אחרון ראשון

דשבורד11:12 29.5.2026עברמסכעות: 958רמה 2.2, רמה 3

דשבורד10:58 29.5.2026עברמסכעות: 1723רמה 1, רמה 1.1, רמה 1.2, רמה 2, רמה 2.1

דשבורד09:55 29.5.2026עברמסכעות: 1957רמה 1, רמה 1.1, רמה 1.2, רמה 2, רמה 2.1, רמה 2.2, רמה 3

פרטי טשן

תאריך: 11:12 29.5.2026מסופל: דשבורד

רמה	תוצאה	מסכעות	זמן (שנ')	הוגש סח"כ	מושלים	שגוי	קליקים כפולים	ילד גלוטן חופיע	הוגש גלוטן	לא הוגש גלוטן	זמן הכנה ממוצע	⊖
רמה 2.2	עבר	445	108	14	13	1	0	3	0	3	8.3	⊕
רמה 3	עבר	513	84	17	16	1	1	2	0	2	5.3	⊕

מנות שהוגשו

רמה 2.2	רמה 3
23	30
9	13
14	17
13	16
1	1

לקוחות שהגיעו

לקוחות שלא קיבלו מנה

הוגש סח"כ

מושלים

שגוי

קשב וחזרתיות

רמה 2.2	רמה 3
0	1

קליקים כפולים

הימנעות מהגשה לילד רגיש לגלוטן

רמה 2.2	רמה 3
3	2
0	0
3	2

ילד גלוטן חופיע

הוגש גלוטן

לא הוגש גלוטן

תוצאת רמה

רמה 2.2	רמה 3
445	513

מסכעות

תוצאה

תכנון ויעילות משימה

רמה 2.2	רמה 3
108	84
8.3	5.3

זמן (שנ')

זמן הכנה ממוצע

הגדרות לוח בקרה

זמן השלב150 שני

מד סבלנות7 שני

סימון מרכיביםכבוי

לכלוךמופעל

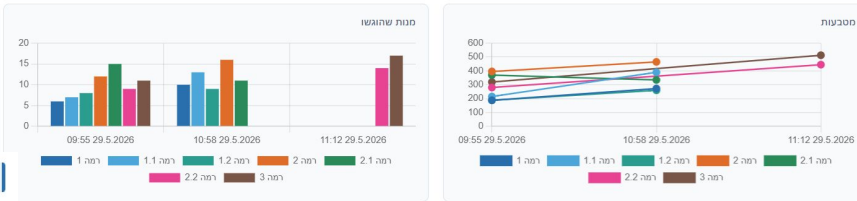
לקוחות במקביל2 / 2

מרכיבים בשלב7 / 7

מנגנון איסוף והצגת הנתונים - Dashboard

גרפים לפי תחומים תפקודיים לאורך זמן

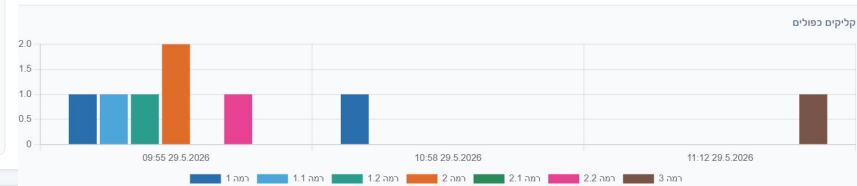
תפוקה ויעילות



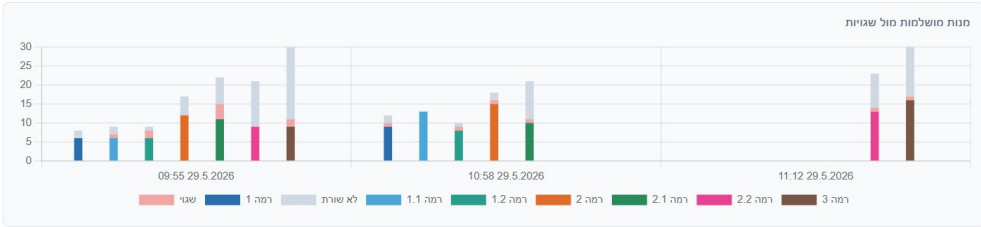
תכנון ויעילות משימה



קשב וחדריות



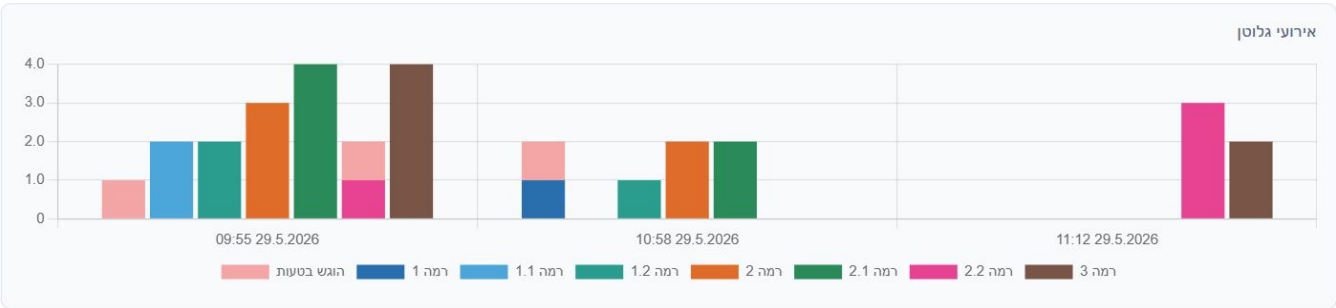
דיוק



מנגנון איסוף והצגת הנתונים - Dashboard

ניהול גלוטן

אירועי גלוטן



הערות על המטופל

הוסף הערות כלליות על המטופל...

הערות על הסשן הנבחר

הוסף הערות לסשן זה...

1. שמירת זהות השחקן (ScoreOfAuthenticatedUser)

```
382 private async Task HandlePostAuth(ButtonType type)
383 {
384     // Populate SessionIdentity so dashboard backend scripts know who is logged in
385     // without needing to access private fields of this class.
386     SessionIdentity.SetFromLogin(displayName, internalUsername, isGuest: false);
387
388     await PostSignInCommon();
389     enabled = true;
390
391     await StartAfterRegisteredLoginAsync(type);
392 }
393 }
```

2 . פורמט תיעוד הנתונים (SessionRecord)

```
38 [System.Serializable]
    5 references
39 public class SessionRecord
40 {
    4 references
41     public string sessionId;
    3 references
42     public string displayName;
    1 reference
43     public string internalUsername;
    3 references
44     public bool isGuest;
    2 references
45     public string sessionDateTimeISO;
    1 reference
46     public string resumeScene;
    4 references
47     public List<LevelAttempt> levels;
48 }
49
```

```
3 [System.Serializable]
    2 references
4 public class LevelControlSettings
5 {
    2 references
6     public int levelDurationSeconds; // actual duration used
    2 references
7     public int levelDurationDefault; // original Inspector value
    2 references
8     public int angerTimeSeconds; // patience meter (7-12)
    2 references
9     public bool markAddedItemsEnabled; // ingredient marking toggle
    2 references
10    public bool dirtEnabled; // dirt toggle
    2 references
11    public int concurrentCustomers; // actual customer limit
    2 references
12    public int concurrentCustomersMax; // max supported for this level (= default)
    2 references
13    public int ingredientCount; // actual ingredient count
    2 references
14    public int ingredientCountMax; // max for this level (= default)
15 }
```

```
17 [System.Serializable]
    6 references
18 public class LevelAttempt
19 {
    4 references
20     public int levelNumber;
    1 reference
21     public bool attempted;
    2 references
22     public bool passed;
    2 references
23     public int coins;
    2 references
24     public int timeToTargetSeconds; // -1 if coin target was never reached
    2 references
25     public int totalServedDishes;
    2 references
26     public int perfectServedDishes;
    1 reference
27     public int incorrectDishes; // derived: totalServed - perfectServed
    2 references
28     public int duplicateIngredientClicks;
    2 references
29     public int glutenChildAppeared;
    2 references
30     public int glutenChildServedByMistake;
    1 reference
31     public int glutenChildHandledCorrectly; // derived: appeared - servedByMistake
    1 reference
32     public float averageDishPrepTimeSeconds; // derived: timeToTarget / perfectServed, -1 if N/A
    2 references
33     public int customersArrived;
    2 references
34     public int playOrder; // 1-based order in which this level was played
```

3 | אחסון הנתונים במשתנים סטטיים (Level**State.cs

```
/// <summary>
/// Holds the state of Level One and Level Three regarding success status
/// and served dishes statistics.
/// </summary>
26 references
public static class LevelOneState
{
    // Whether the level objective was completed successfully
    7 references
    public static bool IsSuccess;

    // Total number of dishes served during the level
    5 references
    public static int TotalServedDishes;

    // Number of dishes that were served with 100% perfect match
    5 references
    public static int PerfectServedDishes;

    4 references
    public static int DuplicateIngredientClicks;
    4 references
    public static int GlutenChildAppeared;
    4 references
    public static int GlutenChildServed;
    3 references
    public static int CustomersArrived;
```

```
21
22
23
24
25
26
27
28
29
30
31 }

public static void Reset()
{
    IsSuccess = false;
    TotalServedDishes = 0;
    PerfectServedDishes = 0;
    DuplicateIngredientClicks = 0;
    GlutenChildAppeared = 0;
    GlutenChildServed = 0;
    CustomersArrived = 0;
}
```

4 . עדכון מצב המשחק הנוכחי

נתון	מתי מתעדכן	סקריפט שמעדכן	סקריפט שקורא
TotalServedDishes	כל הגשת מנה	CustomerManager.cs	SessionDataCollector.cs
PerfectServedDishes	הגשה מושלמת בלבד	CustomerManager.cs	SessionDataCollector.cs
DuplicateIngredientClicks	לחיצה על מרכיב כפול	CustomerManager.cs	SessionDataCollector.cs
CustomersArrived	כניסת לקוח	CustomerManager.cs	SessionDataCollector.cs
GlutenChildAppeared	ילד גלוטן הגיע	CustomerManager.cs	SessionDataCollector.cs
GlutenChildServed	הוגש גלוטן בטעות	CustomerManager.cs	SessionDataCollector.cs
coins	סיום הרמה	ScoreManager.cs	LevelTimerWinLose.cs
passed	סיום הרמה	LevelTimerWinLose.cs	LevelTimerWinLose.cs
timeToTargetSeconds	רגע הגעה ליעד	LevelTimerWinLose.cs	LevelTimerWinLose.cs
הגדרות לוח בקרה	סיום הרמה	/ ControlPanelUI.cs / CustomerMoodTimer.cs CustomerManager.cs	SessionDataCollector.cs

5 . סיום הרמה ותיעוד (Level**TimeWinLose.cs)

```
368 // Record this level's attempt for dashboard export before leaving the scene
369 SessionDataCollector.RecordLevelAttempt(1, timeToTargetSeconds, Mathf.RoundToInt(levelDurationSeconds), Mathf.RoundToInt(originalDurationSeconds));
```

6 . בניית סשן (SessionDataCollector.cs)

```
5 public static class SessionDataCollector
11
12 // Called from EndLevel() in each level's timer script.
13 // timeToTargetSeconds is passed directly from the timer.
14 // levelDurationSeconds is the actual duration used (after +30/-30 adjustments).
15 // originalDurationSeconds is the Inspector default before any adjustments.
16 7 references
17 public static void RecordLevelAttempt(int levelNumber, int timeToTargetSeconds, int levelDurationSeconds, int or
18 {
19     bool passed;
20     int totalServed, perfectServed, duplicateClicks, glutenAppeared, glutenServed, coins, customersArrived;
21     if (levelNumber == 1)
22     {
23         passed = LevelOneState.IsSuccess;
24         totalServed = LevelOneState.TotalServedDishes;
25         perfectServed = LevelOneState.PerfectServedDishes;
26         duplicateClicks = LevelOneState.DuplicateIngredientClicks;
27         glutenAppeared = LevelOneState.GlutenChildAppeared;
28         glutenServed = LevelOneState.GlutenChildServed;
29         customersArrived = LevelOneState.CustomersArrived;
30     }
31     else if (levelNumber == 2)
92
93 coins = ScoreManager.Instance != null ? ScoreManager.Instance.CurrentMoney : 0;
94
95 // Derived metrics - no new runtime tracking needed
96 int incorrectDishes = totalServed - perfectServed;
97 int glutenHandled = glutenAppeared - glutenServed;
98 float avgPrepTime = perfectServed > 0
99     ? (float)timeToTargetSeconds / perfectServed
100     : -1f;
101
102 // Collect control panel settings from live managers at the moment EndLevel fires
103 var controlSettings = new LevelControlSettings
104 {
105     levelDurationSeconds = levelDurationSeconds,
106     levelDurationDefault = originalDurationSeconds,
107     angerTimeSeconds = Mathf.RoundToInt(CustomerMoodTimer_levels.RuntimeSecondsPerStage),
108     markAddedItemsEnabled = ControlPanelUI.MarkAddedItemsEnabled,
109     dirtEnabled = ControlPanelUI.DirtEnabled,
110     concurrentCustomers = CustomerManager.Instance != null ? CustomerManager.Instance.GetCurrentMaxConcurrentCustomers() : 1,
111     concurrentCustomersMax = CustomerManager.Instance != null ? CustomerManager.Instance.GetMaxSupportedConcurrentCustomers() : 1,
112     ingredientCount = LevelIngredientAvailabilityManager.Instance != null ? LevelIngredientAvailabilityManager.Instance.CurrentIngredientCount : 0,
113     ingredientCountMax = LevelIngredientAvailabilityManager.Instance != null ? LevelIngredientAvailabilityManager.Instance.MaxIngredientCount : 0,
114 };
115 }
```

6 . בניית סשן (SessionDataCollector.cs)

```
115 attempts[levelNumber] = new LevelAttempt
116 {
117     levelNumber = levelNumber,
118     attempted = true,
119     playOrder = ++nextPlayOrder,
120     passed = passed,
121     coins = coins,
122     timeToTargetSeconds = timeToTargetSeconds,
123     totalServedDishes = totalServed,
124     perfectServedDishes = perfectServed,
125     incorrectDishes = incorrectDishes,
126     duplicateIngredientClicks = duplicateClicks,
127     glutenChildAppeared = glutenAppeared,
128     glutenChildServedByMistake = glutenServed,
129     glutenChildHandledCorrectly = glutenHandled,
130     averageDishPrepTimeSeconds = avgPrepTime,
131     customersArrived = customersArrived,
132     controlSettings = controlSettings,
133 };
134
135 Debug.Log(
136     $"[SDC] RecordLevelAttempt: level={levelNumber} passed={passed} coins={coins} "
137     + $"timeToTarget={timeToTargetSeconds} totalServed={totalServed} perfectServed={perfectServed} "
138     + $"attempts.Count={attempts.Count}"
139 );
140 }
```

```
176 // Called at session start (login) to clear state from any previous session
177 // 0 references
178 public static void Reset()
179 {
180     attempts.Clear();
181     currentSessionId = null;
182     nextPlayOrder = 0;
183 }
```

```
142 // Called when the player returns to MainMenu (via CloudProgressTracker).
143 // 1 reference
144 public static async Task FinalizeAndExport()
145 {
146     Debug.Log($"[SDC] FinalizeAndExport called: currentSessionId={currentSessionId} attempts.Count={attempts.Count}");
147
148     // No gameplay yet (e.g. MainMenu loaded immediately after login) – nothing to export.
149     if (attempts.Count == 0)
150     {
151         Debug.Log($"[SDC] FinalizeAndExport: returning early – attempts empty (no gameplay yet)");
152         return;
153     }
154
155     // Generate session ID once; reuse it on subsequent end-scene exports within the same play-through.
156     if (currentSessionId == null)
157     {
158         currentSessionId = SessionIdentity.GenerateSessionId();
159     }
160     Debug.Log($"[SDC] FinalizeAndExport: proceeding – sessionId={currentSessionId} levels to export={attempts.Count}");
161
162     var levelList = new List<LevelAttempt>(attempts.Values);
163
164     var record = new SessionRecord
165     {
166         sessionId = currentSessionId,
167         displayName = SessionIdentity.DisplayName ?? "Unknown",
168         internalUsername = SessionIdentity.InternalUsername ?? "unknown",
169         isGuest = SessionIdentity.IsGuest,
170         sessionDateTimeISO = System.DateTime.UtcNow.ToString("o"),
171         resumeScene = "",
172         levels = levelList,
173     };
174
175     await CloudSessionHistory.AppendSession(record);
176 }
```


7. שליחה ל- Supabase

(SupabaseClient.cs)

```
84 // Builds a single row and sends it to Supabase via HTTP POST.
85 // If a row with the same (session_id, level_number) already exists it is updated.
86 // 1 reference
87 private static async Task UpsertRow(SessionRecord record, LevelAttempt level)
88 {
89     var row = new SessionRow
90     {
91         session_id = record.sessionId,
92         display_name = record.displayName,
93         session_date = record.sessionDateTimeISO,
94         level_number = level.levelNumber,
95         passed = level.passed,
96         coins = level.coins,
97         time_seconds = level.timeToTargetSeconds,
98         total_served = level.totalServedDishes,
99         customers_arrived = level.customersArrived,
100         perfect_served = level.perfectServedDishes,
101         duplicate_clicks = level.duplicateIngredientClicks,
102         gluten_appeared = level.glutenChildAppeared,
103         gluten_served = level.glutenChildServedByMistake,
104         level_duration_seconds = level.controlSettings != null ? level.controlSettings.levelDurationSeconds : 0,
105         level_duration_default = level.controlSettings != null ? level.controlSettings.levelDurationDefault : 0,
106         anger_time_seconds = level.controlSettings != null ? level.controlSettings.angerTimeSeconds : 0,
107         mark_added_items_enabled = level.controlSettings != null && level.controlSettings.markAddedItemsEnabled,
108         dirt_enabled = level.controlSettings == null || level.controlSettings.dirtEnabled,
109         concurrent_customers = level.controlSettings != null ? level.controlSettings.concurrentCustomers : 1,
110         concurrent_customers_max = level.controlSettings != null ? level.controlSettings.concurrentCustomersMax : 1,
111         ingredient_count = level.controlSettings != null ? level.controlSettings.ingredientCount : 0,
112         ingredient_count_max = level.controlSettings != null ? level.controlSettings.ingredientCountMax : 0,
113         play_order = level.playOrder,
114     };
115     string json = JsonUtility.ToJson(row);
116     string url = ProjectUrl + Endpoint;
```

```
115     string json = JsonUtility.ToJson(row);
116     string url = ProjectUrl + Endpoint;
117
118     using var request = new UnityWebRequest(url, "POST");
119     request.uploadHandler = new UploadHandlerRaw(System.Text.Encoding.UTF8.GetBytes(json));
120     request.downloadHandler = new DownloadHandlerBuffer();
121
122     // Required headers for Supabase REST API
123     request.SetRequestHeader("Content-Type", "application/json");
124     request.SetRequestHeader("apikey", AnonKey);
125     request.SetRequestHeader("Authorization", "Bearer " + AnonKey);
126
127     // merge-duplicates = upsert: update the existing row on conflict, insert if new
128     request.SetRequestHeader("Prefer", "resolution=merge-duplicates,return=minimal");
129
130     var op = request.SendWebRequest();
131
132     // Await without blocking the main thread
133     while (!op.isDone)
134     {
135         await Task.Yield();
136     }
137
138     if (request.result == UnityWebRequest.Result.Success)
139     {
140         Debug.Log($"[Supabase] Upserted level {level.levelNumber} for '{record.displayName}'");
141     }
142     else
143     {
144         Debug.LogError(
145             $"[Supabase] Upsert failed (level {level.levelNumber}): {request.error} - "
146             + request.downloadHandler.text
147         );
148     }
149 }
```

8. n - Supabase לדשבורד ויזואלי

1 טריגר אוטומטי

```
loadFromSupabase();
```

ברגע שהמרפאה בעיסוק פותחת את קובץ ה-HTML המקומי בדפדפן, מופעלת קריאה אוטומטית לקבלת הנתונים ללא צורך בהתערבות ידנית.

2 GET HTTP קריאה

```
const res = await fetch(url, { headers: { 'apikey': KEY } }); const rows = await res.json();
```

הדפדפן פונה ישירות ל-Supabase באמצעות **Browser API** (ללא תיווך שרת). מתקבל מערך JSON של שורות גולמיות (שורה לכל שלב ששוחק).

3 עיבוד וקיבוץ

```
allSessions = reconstructSessions(rows);
```

המערכת מעבדת את המידע הגולמי:

- קיבוץ רמות לפי session_id.
- חישוב מדדים קליניים נגזרים.
- מיון כרונולוגי לפי סדר המשחק.

4 רינדור מלא

```
render();
```

עדכון דינמי של ה-DOM:

- renderPatientSelector().
- renderCards() ורשימות סשנים.
- בניית גרפים קליניים עם Chart.js.
- טעינת הערות ופילטרים.

פתיחת קובץ ה-HTML

loadFromSupabase()

קבלת JSON גולמי (rows)

reconstructSessions()

אובייקטי סשן מוכנים

render() לעדכון ה-DOM

התאמה ויזואלית



הנחיות

- צמצום ההנחיות.
 - הבלטת הנחיה על גבי הרקע.
 - באמצעות רקע לבן, מרכז והגדלה.
 - "ריקוד" של האימוג'י המתאים
- חציל/ צ'יפס בעמדת הטיגון בארה"ב.



נגישות רכיבים בבוט הדרישה

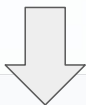
הגדלת אלמנטים על המסך לשיפור הדיוק וההצלחה

התאמת החציל בבוט לחציל במגש



סביבה מותאמת

רקע נקי ומרגיע המותאם לרגישות חושית ולמגבלות עיבוד ויזואלי



תמיכה קולית

- תזכורות בשלבים ההתחלתיים - פעם ראשונה שמופיע ילד רגיש לגלוטן, ובשלב הראשון תזכורת להוצאת החצילים.
- הנחיה קולית ללחוץ על האימוג'י - לטובת הוצאת החציל והציפס ברמה 3.

חיווי קולי מתקדם

הוספנו למשחק רובד שמיעתי קריטי ללמידה מתווכת, וחווית השחקן:

הדמויות נותנות חיווי קולי (חיובי או שלילי), **נוספו חיוויים חדשים** החיווי השמיעתי עוזר למטופל להתמקד במשימה ולקבל **משוב מיידי**. הדיבוב בוצע ברמה מקצועית כדי לייצר סביבה ריאליסטית המעודדת **בקה עצמית** אצל המטופל.

הוספת חיווי שמיעתי בלחיצה על רכיב וכאשר המסך מתלכלך מהרטבים - צליל הדרגתי בהתאמה ל - 3 מצבי לכלוך
צליל בזמן שטיימר הטיגון פועל

שיתוף פעולה עם סטודנטים לתקשורת

דיבוב: אליעד צוקר
הקלטה: אורה דזנאשוילי

תוכנית עבודה: מאי - יוני 2026



שאלונים לשלב הפיילוט



1. שאלון מקדים

מטרה: קבלת רקע על אופי הטיפול כיום בקליניקה.

איסוף מידע על פרופיל המטופלים, מתודולוגיות תיווך קיימות בקליניקה אל מול האוכלוסייה.



2. שאלון מעקב טיפול

מטרה: ניטור מגמות ורציפות טיפולית.

מילוי קצר בסיום כל מפגש המאפשר זיהוי שינויים בתפקוד הקוגניטיבי ותגובת המטופל לתהליך הדרגתי וחזרתי..



3. שאלון סיכום

מטרה: הערכת אימפקט והמלצות להמשך.

ניתוח תוצרים חיוביים, זיהוי פערים שנותרו, ותרומת המשחק להעברת המיומנויות לתפקודי יום-יום.